演算法角色詳解

01-E01 保序、保存與穩定

推導

第一候選包含全部四筆，key序列1,1,2,2有序；但輸入a先c，輸出c先a，違反stable。第二候選缺identity=d，因此即使1,2,2有序，也不符合permutation，還未到穩定性就已失敗。完全正確的輸出須讓key1的b先d、key2的a先c。

結論與檢核

答案[(1,b),(1,d),(2,a),(2,c)]。空輸入回[]，有序與保存都成立；不能為了去重刪records，除非另改問題規格。

01-E02 最小匹配與終止

推導

找8在i=1比較8=8，立即回1。找9依序比較8≠9、3≠9、8≠9、5≠9，共4次；i=5時邊界判斷失敗，不讀A[5]，回NIL。不變量是迴圈開始前A[1..i−1]均不等v，初始化空prefix，確認當前不等才前進，故保持。variant n−i+1在每次進body前非負，之後減1，故不可能無限迭代。

結論與檢核

short-circuit避免i=n+1時評估不存在的A[i]。n=0時i=1≤0為假，零次鍵比較回NIL。不變量證答案位置最小，variant另證終止，兩者不能互相替代。

01-E03 Partial correctness 不足

推導

唯一能終止的執行是偶數分支，它確實返回0，所以「若終止則符合輸出規格」成立。合法輸入n=1卻永不返回，違反total correctness對所有合法n的終止要求。替代程序直接return0，對每個非負n都有限步結束且輸出0。

結論與檢核

原程序partial-correct但不total-correct。偶數測試全部通過不會排除奇數反例；規格中的合法域不能在證明中悄悄縮小。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

演算法角色詳解 （續）

01-E04 路徑與巡迴

推導

直接02成本2，或012成本2；每邊至少1且直接邊為2，沒有成本小於2的0→2路徑。tour固定0後可排其餘3點，共6個方向；因邊無向、反向成本相同，合併成01230成本1+1+1+1=4；01320成本1+2+1+2=6；02130成本2+1+2+1=6。全部三種已列盡，最小4。

結論與檢核

path不要求拜訪所有點與回起點；tour有這兩項額外條件。枚舉證這個四點instance最優，不證一般n可用多項式時間枚舉，因候選數隨n成階乘成長。

01-E05 隨機多數表決的條件

推導

三次獨立錯誤率各≤1/4；多數錯需至少兩錯。錯誤事件對每次錯誤指示都是單調，所以最大值在三率皆1/4：$3(1/4)^2(3/4)+(1/4)^3=9/64+1/64=10/64=5/32$。也可用獨立uniform變數耦合，各次實際錯誤事件由Bernoulli(1/4)支配。串行時間至多3s。

結論與檢核

正確率至少27/32；仍有非零錯誤率。共用亂數使三錯可能完全相關，不能用乘積。search可有三個互不相同而皆正確輸出，沒有共同binary多數語義，需另設checker/選擇規則。

01-E06 同模型的整數門檻

推導

B<A等價$g(n)=n-20\lg n>0$。$g'(x)=1-20/(x\ln 2)$，因此$x\in[2,20/\ln 2]$遞減，其後遞增。$g(2)=-18$，故遞減段無正值；之後只會有一個crossing。lg143約7.1598713，g143約−.1974267；lg144約7.1699250，g144約.6015000。故144是最小持續門檻。

結論與檢核

N=144，143仍B>A，所有n≥144因g單調遞增而B<A。若硬把n=1代入，B1=0<A1=2，但n2又反轉；「最小單點解」與「之後全部成立的門檻」不同。真實單元素算法另有base cost，不能把B1=0當作實測零時間。

演算法角色詳解 （續）

01-E07 輸入bits與硬體單位

推導

n1024時A steps=2×1048576=2097152，時間.02097152s；B steps=40×1024×10=409600，時間.4096s。A硬體快100倍且此n仍不足讓B抵銷。L-bit正整數最大N=2^L-1，逐次迴圈最壞Θ(2^L)；N31是binary11111，L5，做31次。

結論與檢核

操作數少不等於不同機器上秒數少。Θ(N)是對數值N，對編碼長度L是指數階；input-size與cost model必須同時寫出。

01-E08 平行度不是processor截止點

推導

work=6+3=9，span=1+3=4。先完成六個獨立工作，再執行chain，最短T_P=ceil(6/P)+3，因前段要容納6個unit tasks且後段無法平行。P1/2/3/6的T為9、6、5、4，speedup依序1、1.5、1.8、2.25。W/S=9/4=2.25。

結論與檢核

P3已大於2.25，增加到P6仍由5降4；平均平行度限制最大speedup，不構成processor數的收益截止點。所有時間都符合T≥max(9/P,4)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

演算法角色詳解 （續）

原創驗算程式附錄: algorithm_contracts.py

```
"""Finite checks for original chapter examples, not universal proofs."""
from itertools import product,permutations
from fractions import Fraction as F
from math import log2,comb,ceil
from collections import Counter

def first_match(a,v):
    i=0
    while i<len(a) and a[i]!=v:i+=1
    return i+1 if i<len(a) else None

def sort_contract(source,result,stable=False):
    if Counter(source)!=Counter(result):return False
    if any(result[i][0]>result[i+1][0] for i in range(len(result)-1)):return False
    if stable:
        for key in {x[0] for x in source}:
            if [x[1] for x in source if x[0]==key]!=[x[1] for x in result if
x[0]==key]:return False
    return True

if __name__=='__main__':
    count=0
    for n in range(6):
        for a in product(range(3),repeat=n):
            for v in range(3):
                expected=next((i+1 for i,x in enumerate(a) if x==v),None)
                assert first_match(a,v)==expected;count+=1
    src=[(2,'a'),(1,'b'),(2,'c'),(1,'d')]
    assert sort_contract(src,sorted(src,key=lambda r:r[0]),True)
    assert not sort_contract(src,[(1,'b'),(1,'d'),(2,'c'),(2,'a')],True)
    assert not sort_contract(src,[(1,'b'),(2,'a'),(2,'c')])
    def w(a,b):return 1 if (a-b)%4 in (1,3) else 2
    tours=[]
    for p in permutations((1,2,3)):
        if p[0]<p[-1]:
            q=(0,)+p+(0,);tours.append((q,sum(w(a,b) for a,b in zip(q,q[1:]))))
    assert sorted(v for q,v in tours)==[4,6,6]
    assert sum(F(comb(3,k))*F(1,4)**k*F(3,4)**(3-k) for k in (2,3))==F(5,32)
    assert 143-20*log2(143)<0<144-20*log2(144)
    assert [ceil(6/p)+3 for p in (1,2,3,6)]==[9,6,5,4]
    print('FIRST-MATCH finite cases:',count,'; tour costs:',tours)
    print('crossover g143/g144:',143-20*log2(143),144-20*log2(144),'majority error<=5/32')
```

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study